UNIVERSITÉ PARIS 8
Vincennes – Saint-Denis

MASTER 2 INFORMATIQUE
Parcours Ingénierie en Intelligence Artificielle (IIA)

JURISCONSULT AI

Agent IA basé sur le RAG

Conception et implémentation d'un agent d'intelligence artificielle exploitant des documents juridiques pour faciliter la recherche et l'accès à l'information.

Réalisé par
ANTON NELCON Steve
LIMACHE Adrien

Module
Implémentation de cas d'usage

Année universitaire
2026–2027

1. Présentation du projet JurisConsult AI

Dépôt du projet : github.com/an-Steve/Agent-IA-RAG

1.1. Contexte général

JurisConsult AI est un agent d'intelligence artificielle fondé sur une architecture de *Retrieval-Augmented Generation* (RAG) agentique. Il permet d'interroger en langage naturel la jurisprudence civile du Tribunal judiciaire de Paris et de produire des synthèses juridiques appuyées sur des sources documentaires.
Contrairement à un système RAG classique, qui enchaîne généralement une recherche documentaire et une génération de texte, JurisConsult AI s'appuie sur un graphe d'exécution cyclique permettant à l'agent de planifier ses actions, d'utiliser plusieurs outils, d'évaluer les informations collectées et de vérifier les réponses produites.

> **Objectif principal**
> Développer un assistant juridique capable de rechercher des décisions pertinentes, d'analyser leur contenu et de produire des réponses synthétiques, contextualisées et accompagnées de références vérifiables.

1.2. Corpus documentaire

Le système exploite le jeu de données public `antoinejeannot/jurisprudence`, disponible sur Hugging Face. Le fichier `tribunal_judiciaire.parquet` constitue la source documentaire utilisée pour sélectionner les décisions du Tribunal judiciaire de Paris.
Les décisions sont exploitées afin de constituer une base de connaissances permettant la recherche hybride, la lecture des arrêts et la production de réponses fondées sur les éléments retrouvés.

1.3. Architecture de l'agent

L'agent repose sur un graphe cyclique développé avec **LangGraph**, organisé autour de sept nœuds fonctionnels :

Nœud	Rôle
Mémoire	Gestion du contexte de la conversation et conservation d'informations entre sessions.
Planificateur	Sélection de la prochaine action, choix d'un outil ou préparation de la réponse.
Exécuteur d'outils	Lancement des recherches documentaires, lecture des décisions ou recherche externe.
Observateur	Synthèse des éléments et des preuves collectés par l'agent.
Générateur	Rédaction d'une synthèse juridique en distinguant les sources du corpus des sources externes.
Vérificateur	Contrôle de la cohérence entre la réponse proposée et les preuves disponibles, avec possibilité de relancer le cycle.
Finalisation	Validation de la réponse, mise à jour de la mémoire et transmission progressive du résultat à l'interface.

1.4. Outils et mécanisme de recherche

L'agent dispose de trois outils principaux :

Outil	Description
`search_cases`	Recherche hybride combinant BM25 pour la pertinence lexicale et ChromaDB pour la similarité vectorielle.
`read_case`	Lecture d'une décision et extraction d'informations utiles, notamment les acteurs de la procédure, les avocats et certaines données relatives aux dépens.
`external_search`	Consultation de sources externes complémentaires, notamment des ressources juridiques et documentaires.

La recherche hybride repose sur une pondération de **50 % BM25 et 50 % recherche vectorielle**. Les documents sont représentés à l'aide des embeddings `all-MiniLM-L6-v2`, avec un découpage en segments de 1 500 caractères.

1.5. Technologies utilisées

1.5.1. Langages de programmation

Technologie	Rôle
Python 3.10+	Backend, agent LangGraph, préparation et indexation des données.
JavaScript (JSX)	Développement de l'interface React.

1.5.2. Backend et orchestration

Technologie	Rôle
FastAPI et Uvicorn	API REST et transmission des réponses en streaming SSE.
LangChain et LangGraph	Définition des outils et orchestration du graphe agentique à 7 nœuds.
Pydantic	Validation des données et des schémas de réponse.
PyArrow / Parquet	Lecture du corpus documentaire à la demande.

1.5.3. Intelligence artificielle

Technologie	Rôle
Qwen 2.5 1.5B / Ollama	Modèle de langage exécuté localement, sans clé API externe.
SentenceTransformers	Création des embeddings avec `all-MiniLM-L6-v2`.

1.5.4. Recherche documentaire

Technologie	Rôle
ChromaDB	Stockage persistant des vecteurs documentaires.
BM25	Recherche lexicale et indexation des termes.
Recherche hybride	Combinaison BM25 (50 %) et ChromaDB (50 %), suivie d'un reranking structurel.

1.5.5. Frontend et interface utilisateur

Technologie	Rôle
React et Vite	Interface de recherche, lecteur de décisions et suivi de l'agent.
Mermaid	Visualisation du graphe d'exécution de l'agent.
npm	Gestion des dépendances JavaScript.

1.5.6. Sources et stockage des données

Technologie	Rôle
Hugging Face	Dataset `antoinejeannot/jurisprudence`.
Parquet	Stockage des décisions civiles du Tribunal judiciaire de Paris.

1.5.7. Outils de développement

Technologie	Rôle
Git et GitHub	Versionnage et hébergement du code source.
Swagger UI	Documentation interactive de l'API, accessible via `/docs`.
VS Code	Édition et développement du code.

1.5.8. Environnement d'exécution

Élément	Description
Python	Utilisation d'un environnement virtuel `venv` pour isoler les dépendances du backend.
Node.js 18+	Environnement nécessaire à l'exécution du frontend et à la gestion de ses dépendances.
Systèmes d'exploitation	Exécution locale compatible avec Windows, Linux et macOS.

1.5.9. Ports de communication

Service	Port	Utilisation
API FastAPI	8000	Traitement des requêtes et communication avec le backend.
Interface web	5173	Accès à l'application frontend développée avec React et Vite.
Ollama	11434	Communication avec le modèle de langage exécuté localement.

Résultat attendu

Proposer un assistant juridique local capable de rechercher et d'analyser des décisions, de mobiliser plusieurs sources d'information et de produire des synthèses sourcées, tout en limitant les réponses non étayées grâce à un mécanisme de vérification.

2. Installation de l'agent IA

Cette partie présente la mise en œuvre de **JurisConsult AI**, un agent d'intelligence artificielle conçu pour interroger un corpus de décisions de justice et produire des réponses documentées. La démonstration permet d'illustrer le fonctionnement de l'agent, son processus de recherche documentaire et l'exploitation des décisions juridiques à partir de questions formulées en langage naturel.

Vidéo de démonstration

La vidéo suivante présente une démonstration de l'application JurisConsult AI et permet d'observer son fonctionnement en pratique.

Lien : https ://youtu.be/EuvapwFRdoM ?si=UbKcT1USTJY4vPRh

2.1. Préparation de la démonstration

Avant de lancer la démonstration, il est nécessaire d'installer les outils requis, de récupérer le code source du projet, de préparer le corpus documentaire et de construire les index de recherche.
Ces opérations permettent de disposer d'un environnement fonctionnel pour exécuter l'agent IA localement. Le guide suivant détaille les différentes étapes d'installation et de lancement.

2.2. Guide d'installation complet de A à Z

Les étapes suivantes doivent être réalisées dans l'ordre. Les commandes sont prévues pour Windows avec PowerShell, ainsi que pour Linux et macOS lorsque les commandes diffèrent.

Étape 1 — Prérequis système

Avant de commencer, vérifiez que les logiciels suivants sont installés sur votre machine.

Logiciel	Utilité	Téléchargement
Python 3.10+	Exécution du backend et des scripts de préparation.	https ://www.python.org/downloads/pyth
Node.js 18+ et npm	Installation et lancement de l'interface web.	https ://nodejs.org/nodejs.org
Git	Récupération du code source du projet.	https ://git-scm.com/git-scm.com
Ollama	Exécution locale du modèle de langage.	https ://ollama.com/ollama.com

Étape 2 — Cloner le dépôt GitHub

Ouvrez un terminal et récupérez le dépôt contenant le code source de JurisConsult AI.

```
git clone https://github.com/AdrienLIMACHE/jurisconsult-agentic-rag.
cd jurisconsult-agentic-rag
```

Cette opération crée le dossier du projet et permet d'accéder à ses fichiers.

Étape 3 — Créer l'environnement virtuel Python

L'environnement virtuel `venv` permet d'isoler les dépendances Python du projet et d'éviter les conflits avec d'autres installations.

Sous Windows (PowerShell)

```
python -m venv venv
.\venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Sous Linux / macOS

```
python3 -m venv venv
source venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Ces commandes créent l'environnement virtuel, l'activent et installent les dépendances déclarées dans le fichier `requirements.txt`, notamment celles utilisées par FastAPI, LangChain, LangGraph et ChromaDB.

Étape 4 — Installer le modèle de langage avec Ollama

JurisConsult AI utilise le modèle `Qwen 2.5 1.5B`, exécuté localement grâce à Ollama. Assurez-vous qu'Ollama est installé et lancé, puis téléchargez le modèle avec la commande suivante :

```
ollama pull qwen2.5:1.5b
```

Pour vérifier que le modèle est disponible, exécutez :

```
ollama list
```

Le modèle téléchargé pourra ensuite être utilisé par l'agent pour générer ses réponses.

Étape 5 — Télécharger et préparer le corpus juridique

Élément	Description
Plateforme	Hugging Face
Jeu de données	antoinejeannot/jurisprudence
Fichier source	**`tribunal_judiciaire.parquet`**

5.1. Télécharger le fichier source Le corpus juridique utilisé par JurisConsult AI provient du jeu de données https ://huggingface.co/datasets/antoinejeannot/jurisprudence antoinejeannot/jurisprudence disponible sur Hugging Face.
Depuis la racine du projet, exécutez la commande suivante afin de télécharger le fichier `tribunal_judiciai` dans le dossier local `antoinejeannot_jurisprudence/`.

Commande de téléchargement

```
python -c "from huggingface_hub import hf_hub_download; hf_hub_download(rep
```

Résultat attendu : le fichier source est disponible à l'emplacement suivant :

```
antoinejeannot_jurisprudence/tribunal_judiciaire.parquet
```

5.2. Préparer les décisions du Tribunal judiciaire de Paris Après le téléchargement du corpus, exécutez le script de préparation à la racine du projet. Il permet de filtrer les décisions valides du Tribunal judiciaire de Paris et de générer un échantillon destiné à l'indexation documentaire.

Commande de préparation

```
python -m backend.scripts.prepare_data
```

Résultat attendu : le script génère le fichier préparé suivant, qui sera utilisé lors de la construction des index de recherche :

```
data/corpus_sampled.parquet
```

Étape 6 — Construire les index de recherche

L'agent combine une recherche lexicale BM25 et une recherche vectorielle avec ChromaDB. Avant la première utilisation, il faut construire les index documentaires à partir du corpus préparé.
Exécutez la commande suivante à la racine du projet :

```
python -m backend.scripts.build_index
```

Le script découpe les décisions en segments de 1 500 caractères, avec un chevauchement de 200 caractères. Il calcule les embeddings avec `all-MiniLM-L6-v2` et construit les deux index suivants :

- **`chroma_db/`** : base vectorielle persistante ;
- **`data/bm25_index.pkl`** : index de recherche lexicale.

La recherche hybride combine les résultats BM25 et ChromaDB à parts égales (50 % chacun), avant un reranking structurel.
Optionnel : la taille des lots et le délai entre les lots peuvent être ajustés à l'aide de la commande suivante :

```
python -m backend.scripts.build_index --batch-size 10 --cooldown 0.5
```

Étape 7 — Lancer le backend FastAPI

Ouvrez un premier terminal à la racine du projet et vérifiez que l'environnement virtuel est activé. Lancez ensuite le serveur backend :

```
uvicorn backend.main:app --host 0.0.0.0 --port 8000
```

Une fois le serveur démarré, les points d'accès suivants sont disponibles :

Service	Adresse
API REST	`http://localhost:8000`
Documentation Swagger	`http://localhost:8000/docs`
Vérification de santé	`http://localhost:8000/api/health`
Graphe de l'agent	`http://localhost:8000/api/agent/graph`

Étape 8 — Installer et lancer le frontend React

Ouvrez un deuxième terminal. Placez-vous dans le dossier `frontend/`, puis installez les dépendances et démarrez le serveur de développement Vite.

```
cd frontend
npm install
npm run dev
```

Lorsque le serveur frontend est lancé, ouvrez votre navigateur à l'adresse suivante :

http://localhost:5173

2.3. Vérification avant la démonstration

Avant de commencer la démonstration, vérifiez les points suivants :

- L'environnement virtuel Python est activé et les dépendances sont installées.
- Ollama est lancé et le modèle `qwen2.5:1.5b` est disponible.
- Le corpus juridique a été téléchargé et préparé.
- Les index ChromaDB et BM25 ont été construits.
- Le backend FastAPI fonctionne sur le port 8000.
- Le frontend React/Vite est accessible sur le port 5173.

Lorsque ces vérifications sont terminées, vous pouvez utiliser JurisConsult AI depuis votre navigateur et procéder à la démonstration des fonctionnalités de l'agent.

3. Démonstration de l'agent IA

Cette partie présente le fonctionnement de **JurisConsult AI** à travers son interface utilisateur et les principales étapes d'une recherche juridique. L'objectif est d'illustrer comment l'utilisateur peut interroger le corpus de jurisprudence, observer le travail de l'agent et consulter les résultats produits. La démonstration vidéo complète est accessible à l'adresse suivante :

Vidéo de démonstration
https ://youtu.be/EuvapwFRdoM ?si=UbKcT1USTJY4vPRh

3.1. Présentation de l'interface utilisateur



Figure 3.1 — Interface principale de JurisConsult AI

JurisConsult AI propose une interface simple pour chercher dans la jurisprudence du Tribunal judiciaire de Paris (corpus 2024-2025). On saisit une question juridique ou les faits d'un litige dans la barre de recherche, ici « Divorce », et le moteur hybride renvoie les décisions les plus pertinentes.
Les résultats s'affichent sous forme de cartes triées par pertinence : 30 décisions trouvées pour cette requête. Chaque carte indique le rang, la chambre, un score de correspondance (98
Le bouton « Tirer d'autres affaires » affiche une sélection aléatoire, et le lien Judilibre renvoie vers la base officielle de la jurisprudence.

3.3. Le dossier judiciaire

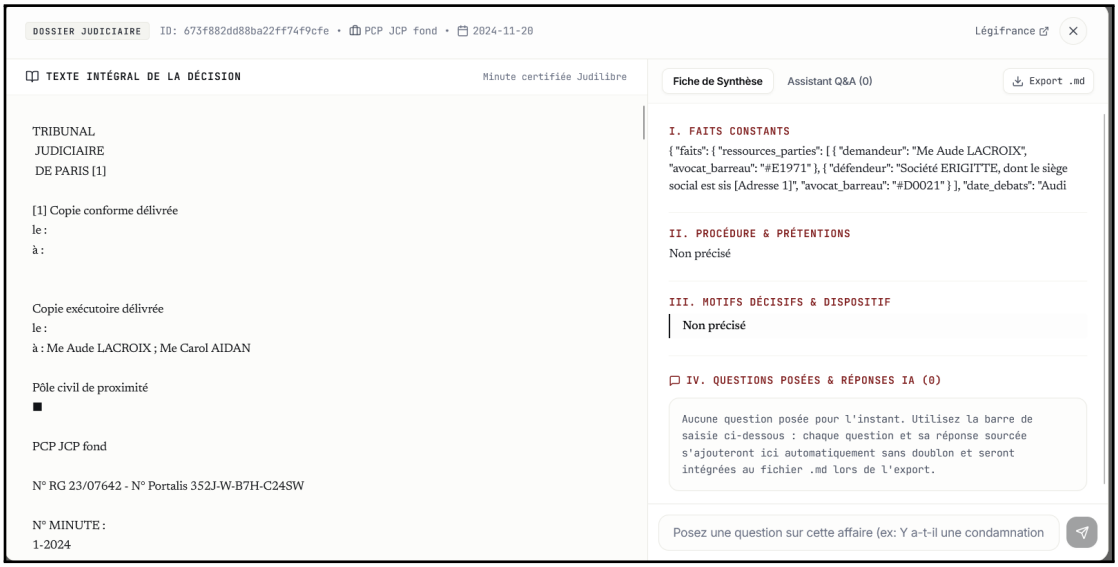


Figure 3.3 — Le dossier judiciaire

Lorsqu'on clique sur « Consulter le dossier » d'une carte, une fenêtre s'ouvre par-dessus la page et affiche la décision choisie en deux parties.
À gauche : le texte intégral de la décision. On y lit la minute certifiée Judilibre, avec l'en-tête du Tribunal judiciaire de Paris, la chambre (ici PCP JCP fond), le numéro de RG, le numéro de minute et le reste du jugement. Le bandeau du haut rappelle l'identifiant du dossier, la chambre et la date, et un lien Légifrance permet d'aller plus loin. La croix en haut à droite ferme la fenêtre.
À droite : l'analyse générée par l'IA, avec deux onglets :
Fiche de Synthèse, qui résume automatiquement l'affaire en quatre parties : I. Faits constants (parties, avocats, date des débats), II. Procédure et prétentions, III. Motifs décisifs et dispositif, IV. Questions posées et réponses IA. Quand une information n'a pas été trouvée dans le texte, la fiche indique « Non précisé ». Assistant Q et A, qui permet de poser une question sur l'affaire dans la barre de saisie en bas (par exemple « Y a-t-il une condamnation ? »). Chaque réponse sourcée s'ajoute automatiquement à la fiche, sans doublon.
Le bouton Export .md permet de télécharger la fiche, avec les questions et réponses, au format Markdown.
On peut ainsi lire la décision et son analyse côte à côte, puis interroger l'IA sur les points qui nous intéressent.

3.4. L'Assisant Q et A

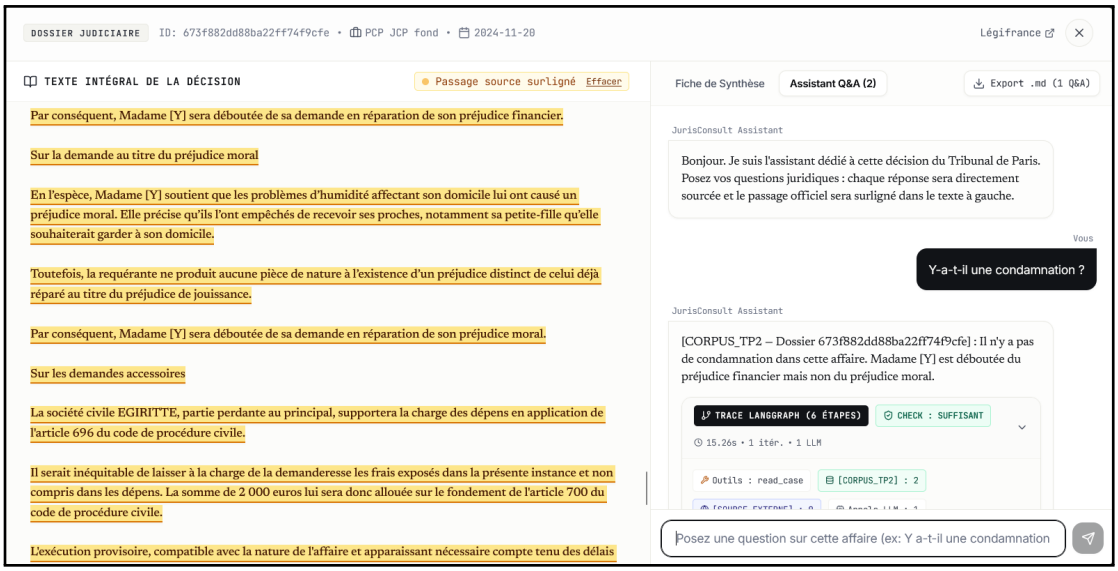


Figure 3.4 — L'Assisant Q et A

Dans l'onglet « Assistant Q etA », on peut poser des questions à l'agent IA sur la décision ouverte. Ici, j'ai demandé : « Y a-t-il une condamnation ? ». L'agent a répondu qu'il n'y en a pas dans cette affaire : la demanderesse est déboutée de sa demande de réparation du préjudice financier et de son préjudice moral.
Chaque réponse est sourcée. L'agent indique qu'elle provient du corpus (balise [CORPUS$_TP2$], $avecl'identifianto$
Sous la réponse, une trace LangGraph détaille le raisonnement de l'agent : 6 étapes, un contrôle « CHECK : SUFFISANT » (la vérification anti-hallucination est validée), un temps de 15,26 secondes, une itération, un appel au LLM, ainsi que l'outil utilisé (read$_case)etlenombredesourcesducorpusmobilisées.Lecomp$
Petite précision : l'agent dit qu'il n'y a pas de condamnation, alors que le passage surligné indique que la société EGIRITTE supporte les dépens et doit verser 2 000 € au titre de l'article 700. Il y a donc bien une condamnation accessoire, et la réponse gagnerait à être plus nuancée (« pas de condamnation sur le fond, mais condamnation aux dépens et à l'article 700 »). C'est un bon exemple de limite d'un petit modèle, que vous pouvez mentionner dans votre présentation.

4. Quelque illustations

Cette partie présente quelques illustrations des fonctionnalités développées dans le cadre du projet JurisConsult AI.

4.1. Architecture et fonctionnement du graphe LangGraph

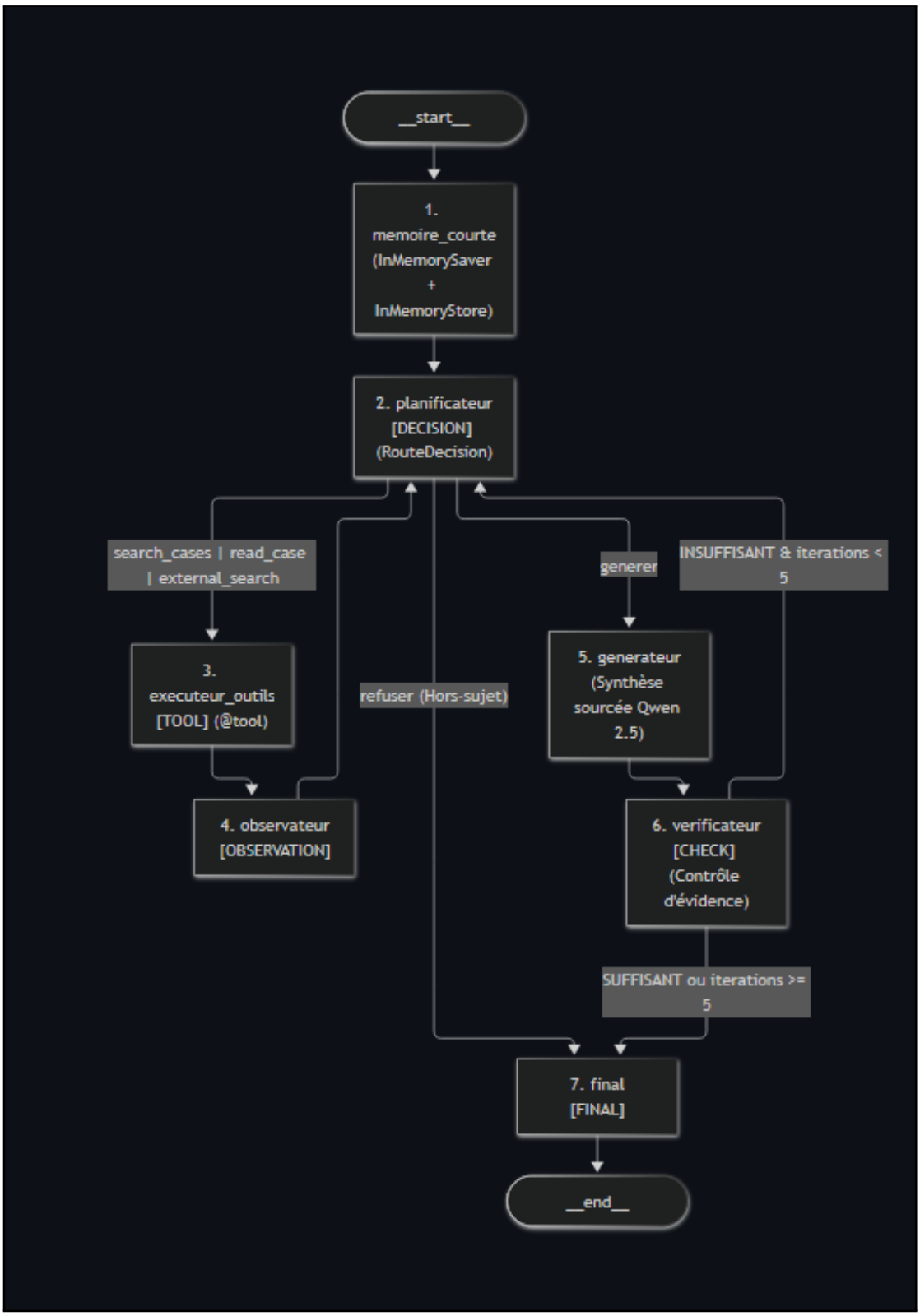


Figure 3.4 — L'Assissant Q et A

Ce schéma représente le graphe LangGraph qui pilote l'agent JurisConsult AI. Contrairement à un pipeline linéaire, cette architecture intègre des boucles de décision permettant à l'agent d'effectuer plusieurs itérations avant de produire sa réponse. Elle se compose de sept nœuds principaux.

1. **`memoire_courte`** Ce nœud constitue le point d'entrée du graphe. Il mobilise la mémoire de conversation (`InMemorySaver`) et la mémoire entre sessions (`InMemoryStore`) afin de conserver le contexte des échanges.

2. **`planificateur`** Ce nœud représente le mécanisme de décision de l'agent. À partir de la question de l'utilisateur, il détermine la prochaine action à effectuer au moyen de `RouteDecision`. Trois possibilités sont envisagées :
   - appeler un outil : `search_cases`, `read_case` ou `external_search` ;
   - passer directement à la génération de la réponse ;
   - refuser une question hors du périmètre juridique prévu, puis terminer le traitement.

3. **`executeur_outils`** Ce nœud exécute l'outil sélectionné par le planificateur. Il peut rechercher des décisions de justice, consulter le contenu détaillé d'un arrêt ou interroger une source juridique externe.

4. **`observateur`** L'observateur analyse et résume les éléments de preuve collectés, puis transmet ces informations au planificateur. Cette première boucle permet à l'agent d'enchaîner plusieurs appels d'outils lorsque les informations disponibles sont encore

4.2. Les 3 couches du projet

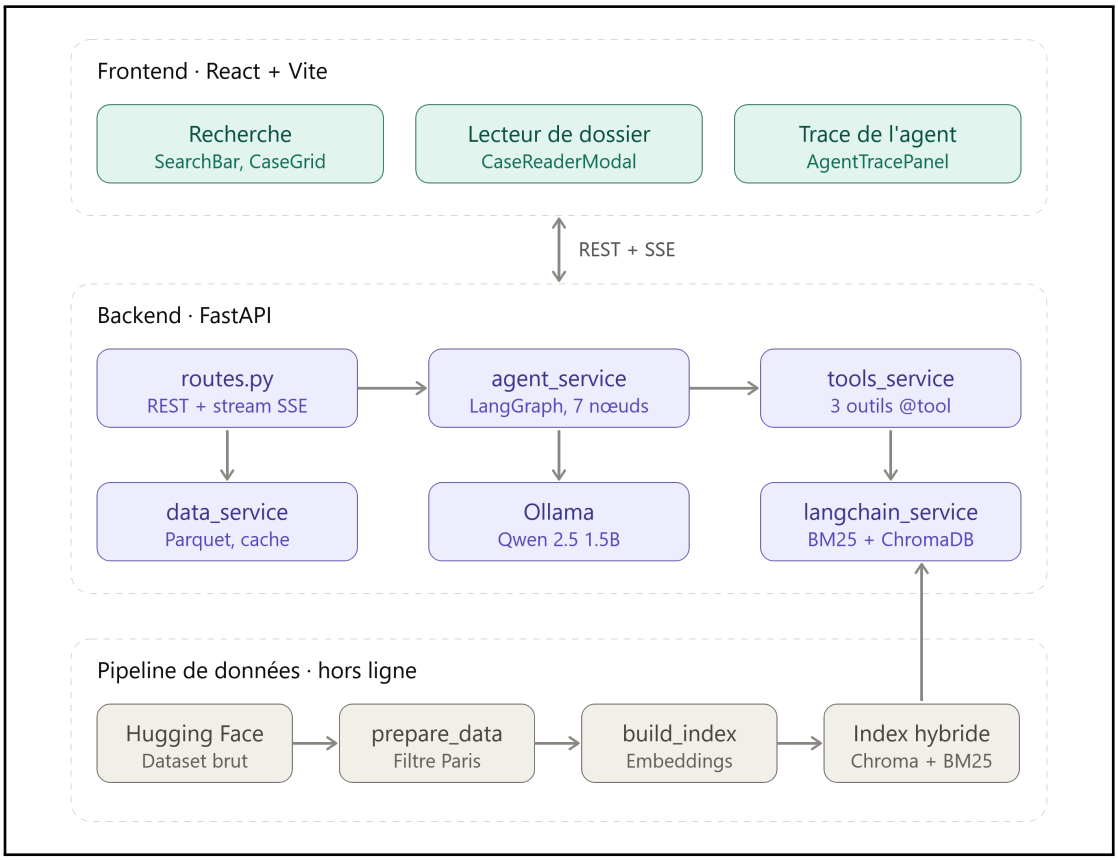


Figure 3.4 — Les 3 couches du projet

Voici le schéma du projet, organisé en trois couches complémentaires.

Frontend (React + Vite). C'est l'interface utilisateur. Elle comprend la recherche et la grille des décisions, le lecteur de dossier avec le texte à gauche et l'assistant à droite, ainsi qu'un panneau affichant la trace de l'agent en temps réel.

Backend (FastAPI). Le fichier `routes.py` expose l'API REST et le flux SSE, interrogés par le frontend. Le module `agent_service` contient le graphe LangGraph à sept nœuds, qui s'appuie sur Ollama et le modèle Qwen 2.5 (1,5 milliard de paramètres) pour générer les réponses.
Le module `tools_service` regroupe les trois outils de l'agent, tandis que `langchain_service` assure la recherche hybride combinant BM25 et ChromaDB. Enfin, `data_service` permet de lire les décisions stockées dans un fichier Parquet.

Pipeline de données (hors ligne). Ce pipeline est exécuté une seule fois avant l'utilisation de l'application. Le dataset est téléchargé depuis Hugging Face, puis le module `prepare_data` filtre les décisions du Tribunal judiciaire de Paris.
Ensuite, le module `build_index` découpe les textes en segments (*chunks*), calcule les représentations vectorielles (*embeddings*) et construit l'index hybride. Le backend réutilise ensuite cet index lors de chaque recherche, comme l'illustre la flèche reliant le pipeline au backend.

4.3. Flux des questions

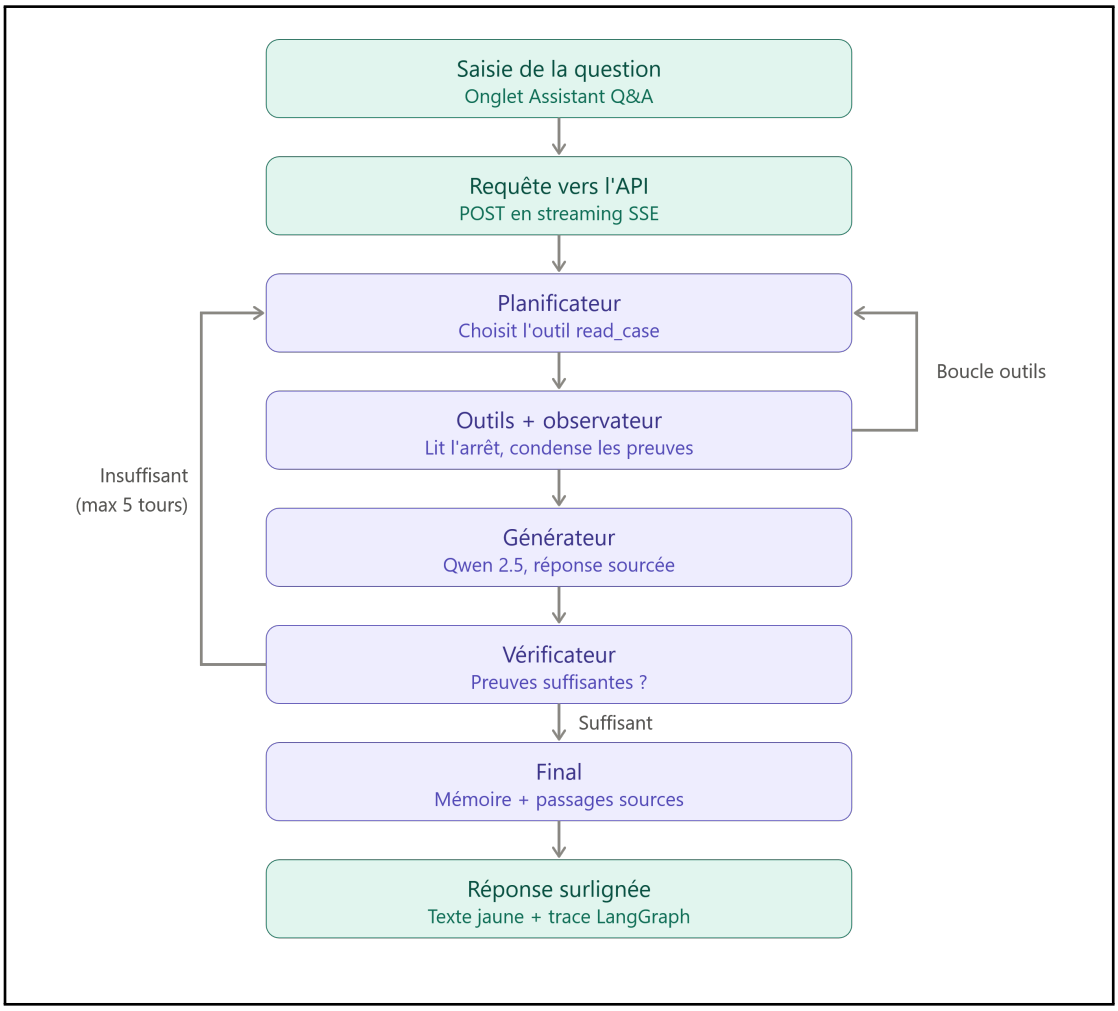


Figure 4.3. — Flux des questions

Cette section présente le parcours d'une question, depuis sa saisie dans l'interface jusqu'à l'affichage de la réponse accompagnée des passages justificatifs surlignés. L'exemple étudié est : *« Y a-t-il une condamnation ? »*.

1. Saisie et envoi de la question L'utilisateur saisit sa question dans l'onglet *Assistant Q&A*. Le frontend transmet ensuite la requête à l'API via un flux SSE (*Server-Sent Events*), permettant d'afficher en temps réel les étapes d'exécution de l'agent.

2. Intervention du planificateur Le planificateur analyse la question et sélectionne l'action appropriée. Dans cet exemple, il choisit l'outil `read_case`, car la demande concerne la décision juridique actuellement ouverte dans l'interface.

3. Exécution de l'outil et observation des preuves L'outil `read_case` consulte le contenu de la décision et extrait les passages pertinents. L'observateur synthétise ensuite les preuves recueillies. Si les éléments disponibles sont insuffisants, le graphe revient au planificateur afin de poursuivre les recherches avec un autre outil.

4. Génération de la réponse Le modèle Qwen 2.5 rédige une réponse structurée à partir des preuves disponibles. La balise `[CORPUS_TP2]` permet d'identifier les informations provenant du corpus documentaire du projet.

5. Vérification de la réponse Le vérificateur examine si la réponse est suffisamment étayée par les preuves collectées. Si le résultat est insuffisant, le graphe relance le cycle de traitement en revenant au planificateur, dans la limite de cinq itérations.

6. Finalisation Lorsque le contrôle est satisfaisant, le nœud final valide la réponse et la sauvegarde en mémoire avec les passages sources utilisés pour justifier les informations fournies.

7. Affichage et surlignage des passages justificatifs L'interface présente la réponse de l'assistant ainsi que la trace LangGraph. Dans le document affiché à gauche, les passages qui justifient la réponse sont surlignés en jaune, ce qui permet à l'utilisateur de retrouver plus facilement les éléments juridiques pertinents.

4.4. La recherche hybride

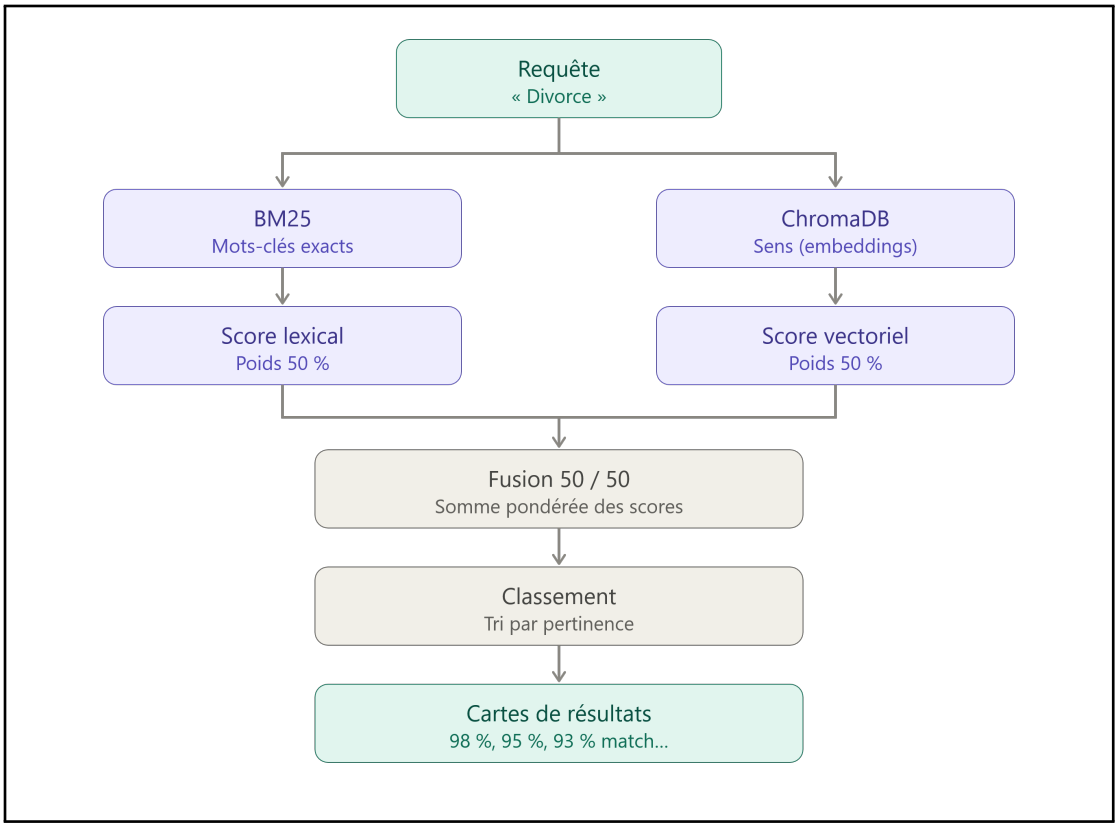


Figure 4.4. — La recherche hybride

Cette section présente le fonctionnement de la recherche hybride de JurisConsult AI à travers l'exemple de la requête *« Divorce »*. Cette approche combine deux méthodes complémentaires : la recherche lexicale BM25 et la recherche sémantique avec ChromaDB.

1. Deux recherches exécutées en parallèle La requête de l'utilisateur est traitée simultanément par deux mécanismes de recherche.

- **Recherche lexicale avec BM25 :** cette méthode recherche les correspondances entre les termes de la requête et ceux des documents. Une décision contenant fréquemment le terme *« divorce »* ou l'expression *« altération définitive du lien conjugal »* peut obtenir un score lexical élevé, selon les termes recherchés et leur importance dans le document.
- **Recherche sémantique avec ChromaDB :** cette méthode compare le sens de la requête à celui des passages documentaires. La requête et les passages sont représentés sous forme de vecteurs numériques (*embeddings*), calculés avec le modèle `all-MiniLM-L6-v2`. Elle peut ainsi retrouver des passages pertinents même lorsque leur formulation diffère de celle de la requête.

2. Pourquoi combiner les deux méthodes ? La recherche BM25 est efficace pour retrouver des termes juridiques précis, mais elle peut manquer certains documents qui utilisent des formulations différentes. À l'inverse, la recherche sémantique permet de retrouver des reformulations pertinentes,

mais les correspondances de sens ne garantissent pas toujours une précision suffisante sur les termes juridiques exacts.
La combinaison des deux approches permet donc de tirer parti de leurs avantages respectifs. Dans la configuration présentée, chaque méthode contribue à hauteur de 50 % au score final.

3. Fusion des scores et affichage des résultats Les scores issus des deux recherches sont combinés en appliquant une pondération de 50 % pour BM25 et de 50 % pour la recherche sémantique. Le score combiné permet ensuite de classer les décisions par ordre de pertinence décroissante.
Les résultats sont présentés sous forme de cartes dans l'interface. Le score de correspondance est notamment affiché sous la forme `98 % match` ou `95 % match`, selon les résultats obtenus par le système.

5. Le code

Cette partie présente les principaux éléments du code source de **JurisConsult AI**, en mettant en évidence l'organisation du projet, les composants essentiels et leur rôle dans le fonctionnement de l'application. L'objectif est d'expliquer comment les différentes technologies sont mises en œuvre pour assurer le traitement des requêtes, la recherche de décisions de justice et la génération de réponses adaptées aux questions juridiques des utilisateurs.
Les extraits de code sélectionnés permettent d'illustrer les mécanismes importants de l'application, depuis la réception d'une question jusqu'à la récupération des documents pertinents et à la production d'une réponse. Une attention particulière est portée à l'articulation entre le frontend, le backend, le système de recherche hybride et l'agent d'intelligence artificielle.
Chaque extrait est accompagné d'explications afin de faciliter la compréhension des choix techniques et de montrer comment les différents modules contribuent au fonctionnement global de JurisConsult AI.

5.1. L'Architecture du Projet

```
jurisconsult-agentic-rag/
├── backend/
│   ├── api/
│   │   └── routes.py               # Endpoints REST (/api/chat, /api/agent/stream SSE, /api/agent/gra
│   ├── scripts/
│   │   ├── prepare_data.py         # Filtrage et création de data/corpus_sampled.parquet
│   │   └── build_index.py          # Construction de l'index hybride ChromaDB + BM25
│   ├── services/
│   │   ├── agent_service.py        # StateGraph LangGraph (7 noeuds, InMemorySaver, InMemoryStore, Ch
│   │   ├── tools_service.py        # 3 outils @tool (search_cases, read_case, external_search)
│   │   ├── langchain_service.py    # Moteur RAG Hybride (BM25 + ChromaDB + Reranking structurel)
│   │   ├── prompt_service.py       # Extracteur déterministe de Fiche d'Identité et prompts QA
│   │   ├── data_service.py         # Lecture Parquet à la demande (Zero-RAM Overhead)
│   │   └── cache_service.py        # Cache JSON persistant des synthèses juridiques
│   ├── config.py                   # Configuration centralisée (Qwen 2.5 1.5B, constantes LangGraph)
│   ├── schemas.py                  # Schémas Pydantic (AgentStep, AgentMetrics, ChatResponse)
│   └── main.py                     # Point d'entrée FastAPI et CORS
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   │   ├── AgentTracePanel.jsx   # Visualisation temps réel des étapes LangGraph et diagramme Merma
│   │   │   ├── CaseReaderModal.jsx   # Lecteur de jugement, surlignage des sources et export Audit
│   │   │   ├── CaseGrid.jsx          # Grille de navigation des décisions
│   │   │   ├── SearchBar.jsx         # Barre de recherche hybride / dense / BM25
│   │   │   └── Navbar.jsx            # Barre de navigation
│   │   ├── services/
│   │   │   └── api.js                # Client HTTP REST et Server-Sent Events (SSE)
│   │   ├── App.jsx                   # Composant racine React
│   │   └── main.jsx                  # Point d'entrée React
│   ├── package.json
│   └── vite.config.js
├── data/
│   └── summaries_cache.json          # Cache pré-calculé des résumés juridiques
└── requirements.txt                  # Dépendances Python du projet
```

Figure 5.1. — L'Architecture du Projet

L'arborescence de **JurisConsult AI** est organisée de manière à séparer les traitements réalisés côté serveur, les composants de l'interface utilisateur et les ressources nécessaires à la recherche juridique. Chaque dossier regroupe des fichiers ayant des responsabilités spécifiques, ce qui facilite la compréhension du code, sa maintenance et l'ajout de nouvelles fonctionnalités.

A. Le backend : traitement et intelligence artificielle

Le dossier `backend/` constitue le cœur fonctionnel de l'application. Développé en Python avec FastAPI, il reçoit les requêtes du frontend, exécute les traitements nécessaires et organise les interactions avec l'agent d'intelligence artificielle.

1. Le dossier `api/` Le fichier `routes.py` définit les points d'accès de l'API REST. La route `/api/chat` permet de traiter les demandes de conversation. La route `/api/agent/stream` utilise les Server-Sent Events (SSE) pour transmettre progressivement les événements produits par l'agent. Enfin, `/api/agent/graph` permet d'accéder aux informations relatives au graphe de l'agent, selon l'implémentation du projet. Ces routes assurent la communication entre l'interface et les services du backend

2. Le dossier `scripts/` Ce dossier regroupe les scripts utilisés pour préparer les données et construire les index de recherche. Le fichier `prepare_data.py` filtre le corpus juridique et produit le fichier `data/corpus_sampled.parquet`. Le fichier `build_index.py` construit les structures nécessaires à la recherche hybride, en associant l'index lexical BM25 à l'index vectoriel de ChromaDB. Cette étape prépare les ressources utilisées ensuite pour retrouver les décisions pertinentes.

3. Le dossier `services/` Le dossier `services/` contient les principaux services métier et les composants liés à l'intelligence artificielle. Il est divisé en plusieurs modules complémentaires :

- **`agent_service.py`** : orchestre l'agent à l'aide de LangGraph et d'un graphe d'état (*StateGraph*) composé de sept nœuds. Il intègre également les mécanismes de sauvegarde d'état et de stockage en mémoire indiqués par `InMemorySaver` et `InMemoryStore`, ainsi que la logique de contrôle identifiée par CHECK.
- **`tools_service.py`** : définit trois outils destinés à être utilisés par l'agent : `search_cases` pour rechercher des décisions, `read_case` pour consulter une décision et `external_search` pour effectuer une recherche externe. Ces outils permettent à l'agent de mobiliser des fonctionnalités adaptées à la demande de l'utilisateur.
- **`langchain_service.py`** : met en œuvre le moteur de recherche augmentée par récupération (*Retrieval-Augmented Generation*, ou RAG). Il combine BM25, qui privilégie les correspondances lexicales, et ChromaDB, qui recherche les passages selon leur proximité vectorielle. Un mécanisme de reclassement structurel (*reranking*) permet ensuite de réordonner les résultats selon les critères définis dans le projet.
- **`prompt_service.py`** : regroupe les instructions adressées au modèle de langage et les mécanismes de génération de réponses aux questions juridiques. Il comprend également un extracteur déterministe de fiche d'identité, destiné à structurer certaines informations selon des règles définies.
- **`data_service.py`** : gère la lecture des données juridiques stockées au format Parquet à la demande. Cette approche évite de charger systématiquement l'ensemble du corpus en mémoire vive et contribue à limiter la consommation de ressources.
- **`cache_service.py`** : assure la gestion du cache JSON des synthèses juridiques. Lorsqu'une synthèse est disponible dans le cache, elle peut être réutilisée plutôt que recalculée, selon la logique prévue par l'application.

4. Les fichiers de configuration et de validation Le fichier `config.py` centralise les paramètres du projet, notamment ceux associés au modèle de langage Qwen 2.5 1.5B et à l'orchestration LangGraph. Le fichier `schemas.py` définit les structures de données validées avec Pydantic, telles que `AgentStep`, `AgentMetrics` et `ChatResponse`. Ces schémas contribuent à harmoniser les données échangées entre les différents composants. Enfin, `main.py` constitue le point d'entrée de l'application FastAPI et configure notamment les échanges inter-origines grâce à CORS.

B. Le frontend : interface utilisateur

Le dossier `frontend/` regroupe l'interface développée avec React et Vite. Il permet à l'utilisateur de formuler une requête, d'explorer les décisions de justice et de visualiser les résultats produits par le backend.

1. Les composants de l'interface

- **`AgentTracePanel.jsx`** : affiche les étapes d'exécution de l'agent en temps réel et propose une visualisation du graphe à l'aide d'un diagramme Mermaid.
- **`CaseReaderModal.jsx`** : présente le contenu d'une décision dans une fenêtre de lecture. Il permet notamment de mettre en évidence les sources et de proposer un export destiné à l'audit, conformément aux fonctionnalités implémentées.
- **`CaseGrid.jsx`** : affiche les décisions sous forme de grille afin de faciliter leur consultation et la navigation entre les résultats.
- **`SearchBar.jsx`** : fournit une interface de saisie des requêtes et permet de sélectionner les modes de recherche prévus : hybride, dense ou BM25.
- **`Navbar.jsx`** : contient la barre de navigation principale et permet d'accéder aux différentes parties de l'interface.

2. Communication et initialisation Le fichier `services/api.js` centralise les échanges entre le frontend et le backend. Il gère les appels HTTP ainsi que la réception des événements SSE nécessaires au suivi progressif de l'agent. Le fichier `App.jsx` constitue le composant racine qui organise les principaux éléments de l'interface, tandis que `main.jsx` initialise l'application React. Enfin, `package.json` décrit les dépendances et les commandes du frontend, et `vite.config.js` contient la configuration de Vite.

C. Données et dépendances du projet

À la racine du projet, le dossier `data/` contient notamment `summaries_cache.json`, un fichier destiné à conserver des résumés juridiques pré-calculés afin de pouvoir les réutiliser. Le fichier `requirements.txt` répertorie les dépendances Python nécessaires à l'environnement backend. Ces éléments complètent l'architecture en assurant la disponibilité des ressources et des bibliothèques requises par l'application.

6. Nos impressions personnelles

Cette partie est consacrée à nos **impressions personnelles** concernant la réalisation du projet **JurisConsult AI**. Elle permet de présenter, pour chacun des membres de l'équipe, les aspects que nous avons appréciés, les difficultés rencontrées ainsi que les éléments qui nous ont semblé moins satisfaisants au cours du développement. À travers ces retours individuels, nous souhaitons mettre en évidence les compétences acquises, les enseignements tirés de cette expérience et les améliorations qui pourraient être envisagées pour de futurs projets. Cette réflexion constitue également l'occasion de prendre du recul sur notre travail, notre collaboration et les défis techniques liés à la conception d'une application reposant sur l'intelligence artificielle.

6.1. ANTON NELCON Steve

La réalisation du projet **JurisConsult AI** a été une expérience enrichissante, qui m'a permis d'approfondir mes connaissances en intelligence artificielle, en recherche documentaire et en développement d'applications. J'ai particulièrement apprécié le fait de travailler sur un cas concret, associant les technologies RAG, les modèles de langage et la recherche de décisions de justice.

Les aspects que j'ai appréciés :

- La découverte et la mise en pratique de technologies d'intelligence artificielle, notamment LangGraph et la recherche hybride combinant BM25 et ChromaDB.
- La conception d'une solution capable de rechercher des informations juridiques et de présenter les résultats de manière structurée.
- Le développement d'une application complète, depuis le traitement des données jusqu'à l'interface utilisateur.
- La possibilité de mettre en pratique les connaissances acquises au cours de ma formation en ingénierie de l'intelligence artificielle.

Les difficultés rencontrées :

- Comprendre et organiser les interactions entre les différents composants du système, notamment le backend, le frontend et l'agent.
- Configurer et combiner plusieurs mécanismes de recherche afin d'obtenir des résultats pertinents.
- Gérer les difficultés techniques liées à l'intégration des différents modules et au bon fonctionnement de l'application.

Les aspects que j'ai moins appréciés :

Certaines phases de développement ont demandé beaucoup de temps, notamment lors du débogage, de l'intégration des composants et de la résolution des problèmes techniques. Ces étapes ont parfois ralenti l'avancement du projet, mais elles m'ont également appris à analyser les erreurs de manière méthodique et à améliorer progressivement la solution développée.

Bilan personnel :

Ce projet m'a permis de renforcer mes compétences techniques et ma capacité à résoudre des problèmes complexes. Il m'a également fait prendre conscience de l'importance de la qualité des données, de la fiabilité des sources et de la vérification des réponses générées par une intelligence artificielle.

6.2. LIMACHE Adrien

La réalisation de ce projet **JurisConsult AI** a constitué une occasion de découvrir les différentes étapes nécessaires à la conception d'une application intégrant de l'intelligence artificielle. Ce travail m'a permis de mieux comprendre l'importance de la collaboration, de l'organisation du code et de l'intégration des différentes fonctionnalités dans une solution cohérente.

Les aspects que j'ai appréciés :

- La participation à un projet concret répondant à un besoin de recherche et de consultation de documents juridiques.
- Le travail en équipe et le partage des connaissances pour faire avancer le projet.
- La découverte des technologies et des méthodes utilisées pour développer une application basée sur l'intelligence artificielle.

Les difficultés rencontrées :

- La compréhension de certains mécanismes techniques et de leurs interactions avec les autres composants de l'application.
- L'intégration des différentes fonctionnalités tout en veillant à conserver un fonctionnement cohérent.
- La résolution des problèmes rencontrés pendant les phases de développement et de test.

Les aspects que j'ai moins appréciés :

Les phases de correction et d'intégration ont parfois été contraignantes, car elles nécessitaient plusieurs vérifications avant d'obtenir le résultat attendu. Elles ont néanmoins contribué à mieux comprendre les exigences d'un projet informatique et l'importance des tests.

Bilan personnel :

Cette expérience m'a permis de développer mes compétences, de mieux appréhender le travail en équipe et de comprendre les enjeux liés à la réalisation d'une application d'intelligence artificielle. Elle constitue une expérience utile pour mes futurs projets universitaires et professionnels.

7. Conclusion générale

Le projet **JurisConsult AI** nous a permis de concevoir une application d'intelligence artificielle dédiée à la recherche et à la consultation de décisions de justice. L'objectif principal était de proposer une solution facilitant l'accès aux informations juridiques grâce à la combinaison de plusieurs technologies, notamment les modèles de langage, la recherche documentaire et les systèmes d'agents intelligents.

Au cours de ce projet, nous avons travaillé sur différentes étapes du développement, depuis la préparation et l'organisation des données juridiques jusqu'à la mise en place d'un moteur de recherche hybride associant BM25 et ChromaDB. Cette approche permet de combiner la recherche par mots-clés avec la recherche sémantique afin de retrouver des décisions potentiellement pertinentes pour une requête donnée. L'intégration de LangGraph permet également de structurer les étapes de traitement de l'agent et de coordonner l'utilisation des outils mis à sa disposition.

La réalisation de l'interface utilisateur avec React et Vite, ainsi que le développement du backend avec FastAPI, nous ont permis de mettre en œuvre une architecture séparant les traitements métier de la présentation graphique. Les mécanismes de suivi des étapes de l'agent et de consultation des décisions contribuent à rendre le fonctionnement de l'application plus lisible pour l'utilisateur.

Sur le plan technique, ce projet nous a permis d'approfondir nos connaissances en intelligence artificielle générative, en systèmes RAG, en recherche d'information et en développement d'applications web. Nous avons également été confrontés à plusieurs défis liés à l'intégration des composants, à la pertinence des résultats, à la gestion des données et à la nécessité de vérifier les informations produites par le modèle. Ces difficultés nous ont permis de mieux comprendre les exigences associées à la conception d'une solution d'intelligence artificielle fiable et cohérente.

Toutefois, JurisConsult AI doit être considéré comme un outil d'aide à la recherche documentaire et non comme un substitut à l'analyse d'un professionnel du droit. La pertinence des résultats, la qualité des sources et l'exactitude des réponses doivent rester des priorités. Une vérification humaine demeure nécessaire avant toute utilisation des informations dans un contexte juridique réel.

Enfin, plusieurs perspectives d'amélioration peuvent être envisagées, notamment l'enrichissement du corpus juridique, l'amélioration de l'évaluation des résultats de recherche, le renforcement de la vérification des réponses générées et l'optimisation des performances de l'application. L'ajout de nouvelles fonctionnalités et l'amélioration continue de l'interface pourraient également rendre l'outil plus pratique et plus accessible.

En conclusion, **JurisConsult AI représente une mise en pratique des connaissances acquises en ingénierie de l'intelligence artificielle** à travers un projet combinant recherche documentaire, génération de réponses et orchestration d'un agent intelligent. Cette expérience nous a permis de mieux comprendre les possibilités offertes par les technologies d'IA appliquées au domaine juridique, mais aussi leurs limites et les précautions nécessaires à leur utilisation.

Rapport de projet

Réalisé par **ANTON NELCON Steve- LIMACHE Adrien**
Étudiants en Master 2 Informatique
Parcours Ingénierie en Intelligence Artificielle (IIA)
Université Paris 8 Vincennes–Saint-Denis